

Comparing Machine Learning Models for Monthly Stock Return Prediction

Course Project Report

Momen Ismail

Matriculation Number: 50307564

Institutional Email:

s73misma@uni-bonn.de

Machine Learning for Finance

University of Bonn

Instructor: Prof. Ivan Gufler

GitHub Repository

<https://github.com/Momen-Ismail/stock-return-ml-project>

2 August 2026

Abstract

This project compares six models for predicting next-month stock excess returns: a historical mean benchmark, OLS-3, Partial Least Squares, Elastic Net, Random Forest, and Gradient Boosting. I use a monthly panel with 211,059 stock-month observations, 656 tickers, and 484 predictors from February 1990 to December 2025. Hyperparameters are selected with 15 annual expanding-window folds spanning 2005–2019, and the selected specifications are evaluated on the 2020–2025 test sample. Random Forest performs best and reduces monthly squared forecast error by about 0.75% relative to the historical mean. OLS-3 provides a very small improvement, Elastic Net reduces all coefficients to zero, and PLS and Gradient Boosting perform worse than the benchmark. The main conclusion is that machine learning can provide a modest improvement, but the result is not stable across years and does not by itself imply a profitable trading strategy.

Keywords: stock return prediction, machine learning, out-of-sample evaluation, Random Forest, asset pricing

1 Introduction

Predicting stock returns is a difficult problem. Monthly returns are noisy, market conditions change, and relationships that appear useful in one period may disappear in another. At the same time, investors and researchers have a large amount of information available, including price history, accounting variables, market conditions, and macroeconomic indicators. This creates a natural setting for comparing traditional statistical models with machine learning methods.

Machine learning is useful in empirical asset pricing for three main reasons. First, expected-return estimation is a prediction problem: information known at time t is used to predict returns at time $t + 1$. Second, the number of possible predictors is large and many variables are correlated. Third, the relationship between characteristics and future returns may be nonlinear and may depend on interactions between firm-level and aggregate variables. These points motivate regularization, dimension reduction, and tree-based methods (Gu et al. 2020).

The research question of this project is:

Can machine learning models predict next-month stock excess returns more accurately than a simple historical-mean benchmark?

I compare six models under the same data and sample design. The historical mean is the benchmark. OLS-3 is a simple linear model using size, book-to-market, and 12-month momentum. PLS and Elastic Net use all 484 predictors but control the high dimensionality in different ways. Random Forest and Gradient Boosting allow nonlinear relationships and interactions.

The comparison is designed to be realistic for financial prediction. I do not use random train-test splits. Hyperparameters are selected directly from 15 annual expanding-window folds in the 1990–2019 development period. The fixed reference specifications are candidates in the same grids, not competitors in a second selection stage. The selected models are evaluated on the 2020–2025 test period. This follows the main lesson from the forecasting literature: a model should be judged on genuinely future observations and against a clear benchmark (Welch and Goyal 2008).

The project is mainly a comparison exercise. I do not propose a new algorithm, and I do not claim that the best model creates a profitable trading strategy. The goal is to see which model generalizes best, where the models fail, and whether greater complexity is actually useful in this data set.

The rest of the paper is organized as follows. [Section 2](#) describes the data and preprocessing. [Section 3](#) explains the models and the comparison design. [Section 4](#) presents the tuning and test results. [Section 5](#) discusses the main limitations and what I would do differently. [Section 6](#) concludes.

2 Data

2.1 Sources and observation unit

The observation unit is a stock-month. The stock universe is based on a locked S&P 500-style list that contains current and historical tickers available in the project inputs. This is not a perfect point-in-time index-constituent data set, but the same locked universe is used throughout the pipeline.

Daily stock prices and trading volume come from Yahoo Finance. Daily S&P 500 and VIX data are used for market and volatility variables. Monthly Fama–French data provide the risk-free rate. Annual Compustat data provide accounting variables and industry classifications. Eight aggregate predictor states are taken from the Welch–Goyal data set.

Daily histories begin in 1987 so that longer-window variables can be calculated before the main sample starts. The final model-ready panel covers February 1990 to December 2025.

2.2 Prediction target

For stock i in month t , the target is the next-month excess return:

$$y_{i,t+1} = r_{i,t+1} - RF_{t+1}, \quad (1)$$

where $r_{i,t+1}$ is the stock return in the next calendar month and RF_{t+1} is the corresponding risk-free rate.

The target is created only when the next observation for the same ticker is the immediately following calendar month. This avoids treating a return over a gap of several months as a one-month return. The target itself is kept in its raw return units and is not winsorized.

2.3 Timing and leakage controls

The main timing rule is simple: a row dated month t may only use information available by the end of month t .

Stock-price characteristics use price history through month t . Compustat requires an additional timing rule because the fiscal year-end date is not the same as the public reporting date. I therefore treat annual accounting data as available six months after fiscal year-end and use the latest report available for each stock-month. Very old accounting observations are not carried forward indefinitely. The Welch–Goyal variables are lagged by one month. Therefore, a stock-month in month t uses the aggregate state from month $t - 1$. Together with the exact next-month target rule, these choices reduce look-ahead bias.

2.4 Preprocessing

Missing values in the 45 continuous firm characteristics are replaced with the cross-sectional median of the same month. This uses information from other firms in that month, but not from future months. The continuous characteristics are then winsorized within each month at the 1st and 99th percentiles. This limits the influence of extreme predictor values while keeping the target unchanged.

The three binary characteristics remain binary. Market and VIX variables, Welch–Goyal variables, and SIC2 indicators are not cross-sectionally winsorized. The 360 interaction variables are formed from the processed continuous characteristics and the eight aggregate states.

2.5 Predictors and final sample

The final data contain 484 predictors:

- 45 continuous stock and accounting characteristics;
- 3 binary stock characteristics;
- 4 market and VIX variables;

- 8 lagged Welch–Goyal variables;
- 64 SIC2 industry indicators;
- 360 characteristic–macro interactions.

The continuous characteristics cover size, valuation, profitability, investment, leverage, liquidity, momentum, reversal, volatility, beta, and trading activity. The interactions allow the effect of a firm characteristic to depend on the aggregate economic state.

The final panel contains 211,059 observations and 656 tickers. It has no missing predictor values, infinite values, or duplicated ticker-month observations.

Table 1: Chronological sample design

Sample	Observations	Months	Tickers	Start	End
Development	166,447	359	613	1990-02	2019-12
Test	44,612	72	641	2020-01	2025-12

The development sample is used for expanding-window tuning and final refitting. The initial fold estimates models through December 2004 and validates on 2005; the window then expands annually until the final 2019 validation fold. The test sample is not loaded by the tuning or configuration-selection stages.

3 Methodology

3.1 Prediction framework

The purpose of the models is to use information known at the end of month t to predict the excess return realized in month $t + 1$. I write the forecasting problem as

$$r_{i,t+1}^e = \mathbb{E}_t \left[r_{i,t+1}^e \right] + \varepsilon_{i,t+1}, \quad \mathbb{E}_t \left[r_{i,t+1}^e \right] = g(\mathbf{x}_{i,t}), \quad (2)$$

where $r_{i,t+1}^e$ is the next-month excess return of stock i , $\mathbf{x}_{i,t}$ is the information available at month t , and $\varepsilon_{i,t+1}$ is the part of the return that is not predicted by the model.

The models differ in how they estimate the function $g(\cdot)$. Some impose a simple linear relationship, while others allow regularization, dimension reduction, nonlinearities, and interactions. I evaluate all models using the same target, chronological samples, and performance measures so that their results are directly comparable.

3.2 Model choice and rationale

I chose six models that represent different levels and types of model complexity. The purpose is not only to find the best-performing model, but also to test whether additional complexity is actually useful for this prediction problem.

Table 2: Models included in the comparison

Model	Reason for inclusion
Historical Mean	A simple constant benchmark. It shows whether the other models provide any improvement over the average return observed in the available estimation sample.
OLS-3	A small and interpretable linear model using size, book-to-market, and 12-month momentum. It provides a comparison between a simple economic model and the high-dimensional methods.
Partial Least Squares	A dimension-reduction model that replaces the 484 correlated predictors with a small number of supervised components.
Elastic Net	A regularized linear model that can shrink unstable coefficients and set some of them exactly to zero.
Random Forest	A tree ensemble that can capture nonlinear relationships and interactions while reducing the instability of an individual tree through averaging.
Gradient Boosting	A second tree ensemble that fits shallow trees sequentially, with each new tree trying to correct errors made by the previous trees.

Historical Mean

The historical mean is the main benchmark. It ignores all stock characteristics and assigns the same prediction to every stock. During each training or validation exercise, the mean is calculated using only the permitted estimation sample. For the final test, it is calculated using the complete February 1990–December 2019 development sample.

This is an important benchmark because a machine-learning model is only useful for this project if it improves on a simple constant forecast.

OLS-3

OLS-3 uses three predictors:

- firm size;
- book-to-market;
- 12-month momentum.

I chose these variables because they are well-known and economically interpretable stock characteristics (Fama and French 1992; Jegadeesh and Titman 1993). The model provides a useful middle point between the historical mean and the 484-predictor models.

It also allows me to examine whether a small set of established predictors can perform as well as, or better than, much more complex methods.

Partial Least Squares

Partial Least Squares (PLS) is included because the predictor set is high-dimensional and strongly correlated. PLS constructs a small number of latent components using information from both the predictor matrix and the target. It therefore differs from an unsupervised method such as ordinary Principal Component Analysis, which constructs components without using the target variable (Wold et al. 1984).

The main hyperparameter is the number of PLS components. Too few components may remove useful information, while too many components may reproduce noise from the original predictor set.

Elastic Net

Elastic Net is included as a high-dimensional linear model. It combines two types of regularization:

- the L_1 penalty, which can set coefficients to zero;
- the L_2 penalty, which stabilizes coefficients when predictors are strongly correlated.

The parameter α controls the overall strength of regularization, while the l_1 -ratio determines the balance between the two penalties (Tibshirani 1996; Zou and Hastie 2005).

This model is useful because it tests whether the predictive information can be represented through a sparse or heavily regularized linear relationship.

Random Forest

Random Forest fits many regression trees to bootstrap samples and averages their predictions (Breiman 2001). Each split considers only a random subset of predictors.

The random sampling of both observations and predictors reduces the correlation between the trees. Averaging then reduces the variance of an individual tree. The model can still represent nonlinearities and interactions without requiring these relationships to be specified in advance.

The Random Forest specification also uses out-of-bag observations. I use the out-of-bag result only as an additional development-sample diagnostic. It is not treated as final test evidence and is not used instead of the chronological validation and test samples.

Gradient Boosting

Gradient Boosting also combines regression trees, but it does so sequentially. Each new tree is fitted to improve the errors remaining from the existing ensemble.

The main decisions are:

- the number of trees;
- the learning rate;
- the maximum depth of each tree.

A small learning rate makes each tree contribute only a limited correction. Shallow trees restrict the complexity of each individual learner. The number of trees determines how many sequential corrections the model is allowed to make.

3.3 Model-specific scaling and implementation choices

The continuous firm characteristics were already median-imputed and winsorized during data construction. For model estimation, I make an additional scaling decision.

The predictors used by OLS-3, PLS, and Elastic Net are standardized using the mean and standard deviation of the relevant estimation sample. The same fitted transformation is then applied to the corresponding validation or test sample.

Standardization is important for PLS and Elastic Net because their components and penalties depend on the scale of the predictors. I also standardize OLS-3 for consistency, although standardization does not change the fitted values of an unregularized OLS regression with an intercept.

Random Forest and Gradient Boosting are not standardized. Tree splits depend on the ordering of predictor values rather than their measurement units, so rescaling the variables is unnecessary.

Both tree models use a fixed random seed of 42 so that the results can be reproduced.

3.4 Tuning design

I do not use random cross-validation. A random split would mix earlier and later market observations and could allow future market conditions to influence model selection.

Instead, hyperparameters are selected using 15 annual expanding-window folds inside the February 1990–December 2019 development sample:

- the first fold trains through December 2004 and validates on 2005;
- the training window then expands by one year for each new fold;
- the final fold trains through December 2018 and validates on 2019.

For each candidate specification, I calculate monthly MSE in every validation year and then average the 15 annual results:

$$\text{CV-MSE}(\lambda) = \frac{1}{15} \sum_{k=1}^{15} \text{MSE}_{\text{month},k}(\lambda), \quad (3)$$

where λ represents the hyperparameter combination.

The tuning stage examines:

- the number of components for PLS;
- α and the l_1 -ratio for Elastic Net;
- the number of trees for Random Forest;
- the number of trees, learning rate, and maximum depth for Gradient Boosting.

I use monthly MSE as the tuning criterion because the number of stocks differs across months. Averaging errors within month first prevents months with more observations from receiving greater weight.

For PLS and Elastic Net, a new StandardScaler is fitted within every training fold and then applied to that fold’s validation year. Random Forest and Gradient Boosting remain unscaled. Thus no preprocessing parameter is learned from a validation year or from the test sample.

3.5 Candidate configurations and selection

The original fixed specifications remain useful descriptive baselines. They are included once in the candidate grids, however, and do not enter a separate fixed-versus-tuned decision. Within each family, I select the candidate with the exact minimum unrounded average monthly MSE. Deterministic grid order is used only when stored candidate values are exactly tied.

Table 3: Candidate grids and expanding-window selections

Model family	Complete candidate grid	Original fixed candidate	Selected configuration	Average MSE
PLS	Components: 1, 2, 3, 5, 20	20 components	2 components	0.009524326361922
Elastic Net	$\alpha \in \{0.01, 0.015, 0.02\}$, l_1 -ratio $\in \{0.75, 0.85, 0.90\}$, plus fixed	$\alpha = 0.0001$, ratio = 0.50	$\alpha = 0.015$, ratio = 0.85	0.009280693
Random Forest	Trees: 100, 200, 300; square-root feature sampling; leaf size 20	100 trees	300 trees	0.010110163460114
Gradient Boosting	Trees: 100, 200, 300; rates: 0.01, 0.3; depths: 1, 2	100 trees, rate 0.01, depth 2	100 trees, rate 0.01, depth 1	0.009548724324135

Notes: Random Forest candidates also use bootstrap sampling, `n_jobs=-1`, and seed 42. Gradient Boosting uses seed 42. The fixed column is descriptive only; selection uses the minimum average MSE over all 15 folds.

The final Random Forest specification is therefore:

- 300 trees;
- square-root feature sampling at each split;
- minimum leaf size of 20;

- bootstrap sampling;
- out-of-bag diagnostics;
- random seed 42.

The final Gradient Boosting specification is:

- 100 trees;
- learning rate 0.01;
- maximum tree depth 1;
- random seed 42.

The selection code reads development observations only. It does not load the 2020–2025 test sample.

3.6 Final estimation and test procedure

After the configurations are selected, I refit each model on the complete February 1990–December 2019 development sample.

The final procedure is:

1. evaluate all candidates in the 15 expanding 2005–2019 folds;
2. select each family’s exact minimum average monthly MSE;
3. refit each selected configuration on the complete 1990–2019 development sample;
4. evaluate each fitted model on the January 2020–December 2025 test sample.

The test contains 44,612 stock-month observations over 72 months.

3.7 Status of the revised analysis

This report presents a revised analysis produced after the originally submitted implementation and its 2020–2025 outcomes had been reviewed. The revised model-selection code remains technically isolated from the test data: Steps 02 and 03 load only the development period. Nevertheless, the test period is not a pristine never-observed holdout for the revision process. The current results should therefore be read as a transparent corrected analysis, not as evidence from a newly opened test sample. Any further model development would require a new future holdout period.

3.8 Evaluation metrics

The primary evaluation measure is monthly mean squared error:

$$\text{MSE}_{\text{month}} = \frac{1}{T} \sum_{t=1}^T \left[\frac{1}{N_t} \sum_{i=1}^{N_t} (y_{i,t+1} - \hat{y}_{i,t+1})^2 \right]. \quad (4)$$

The squared errors are first averaged across stocks within each month and then across months. This gives every month the same weight, even when the number of stocks differs.

The main benchmark-relative measure is monthly out-of-sample R^2 :

$$R_{\text{OOS,month}}^2 = 1 - \frac{\text{MSE}_{\text{month,model}}}{\text{MSE}_{\text{month,HM}}}. \quad (5)$$

A positive value means that the model reduces average monthly squared forecast error relative to the historical mean. A negative value means that it performs worse than the benchmark.

The benchmark in this project is the historical mean estimated from the permitted estimation sample. It is not a zero-return forecast. Therefore, the out-of-sample R^2 measures improvement relative to an estimated constant expected-return forecast.

I also report:

- monthly RMSE;
- pooled RMSE;
- pooled MAE;
- pooled out-of-sample R^2 ;
- prediction-target correlation.

The monthly measures give equal weight to months, while the pooled measures give equal weight to individual stock-month observations. I treat the monthly measure as the main criterion and the pooled measures as supporting evidence.

Correlation is undefined for constant predictions. This applies to the historical mean and to the final Elastic Net model, which produces an effectively constant prediction.

3.9 Scope of the comparison

The model ranking is descriptive. I do not conduct a formal statistical test showing that the leading model is significantly better than every competing model.

I also do not construct investment portfolios or calculate turnover, transaction costs, Sharpe ratios, or cumulative trading returns. The methodology therefore compares statistical forecast accuracy, not investment profitability.

Finally, I do not create an additional ensemble that combines the predictions of all six models. Random Forest and Gradient Boosting are themselves ensemble methods, but the final comparison evaluates each of the six approaches separately.

4 Results

4.1 Overall final-test performance

After expanding-window tuning and direct configuration selection, I refitted the models on the complete February 1990–December 2019 development sample and evaluated them on January 2020–December 2025.

Table 4 reports the final results. The models are ranked by monthly MSE, which is the main evaluation criterion. The two out-of-sample R^2 columns are reported in percent to make the relatively small differences easier to read.

Table 4: Final test performance, 2020–2025

Rank	Model	Monthly MSE	Monthly RMSE	Monthly OOS R^2 (%)	Pooled OOS R^2 (%)	Correlation
1	Random Forest	0.022200	0.148996	0.7482	0.7149	0.0947
2	OLS-3	0.022360	0.149534	0.0301	0.0296	0.0313
3	Elastic Net	0.022367	0.149556	0.0000	0.0000	–
4	Historical Mean	0.022367	0.149556	0.0000	0.0000	–
5	Gradient Boosting	0.022378	0.149593	-0.0493	-0.0553	0.0023
6	PLS	0.022509	0.150032	-0.6363	-0.6579	0.0270

Notes: Models are ranked by monthly MSE. The reported out-of-sample R^2 values have been multiplied by 100 and are therefore shown as percentages. Monthly OOS R^2 compares monthly MSE with the historical-mean benchmark. Pooled OOS R^2 uses all stock-month squared errors. Correlation is undefined for constant predictions.

Random Forest ranks first according to the prespecified monthly MSE criterion. Its monthly out-of-sample R^2 is 0.7482%. This means that the model reduces average monthly squared prediction error by approximately 0.75% relative to the historical mean. It does not mean that the model predicts a return of 0.75%.

The pooled out-of-sample R^2 is similar at 0.7149%. The monthly measure gives every month equal

weight, while the pooled measure gives every stock-month observation equal weight. The similar values show that the result is not driven only by months containing more stocks.

OLS-3 ranks second, but its improvement is very small. Its monthly out-of-sample R^2 is only 0.0301%. Nevertheless, this is an important comparison because OLS-3 uses only size, book-to-market and 12-month momentum. A simple three-variable model therefore remains competitive with the much larger models.

Elastic Net is effectively tied with the historical mean. The selected penalty shrinks all 484 coefficients to zero, so the model produces a constant prediction. I interpret this as a valid negative result rather than a coding error. Under the selected penalty and chronological validation design, Elastic Net did not find stable linear effects that improved on the benchmark.

Gradient Boosting improves substantially relative to the earlier depth-2 specification, but its selected depth-1 model remains slightly below the historical-mean benchmark. Its prediction-target correlation is positive but close to zero. PLS performs worst by monthly MSE. These results show that additional model complexity does not automatically improve out-of-sample performance.

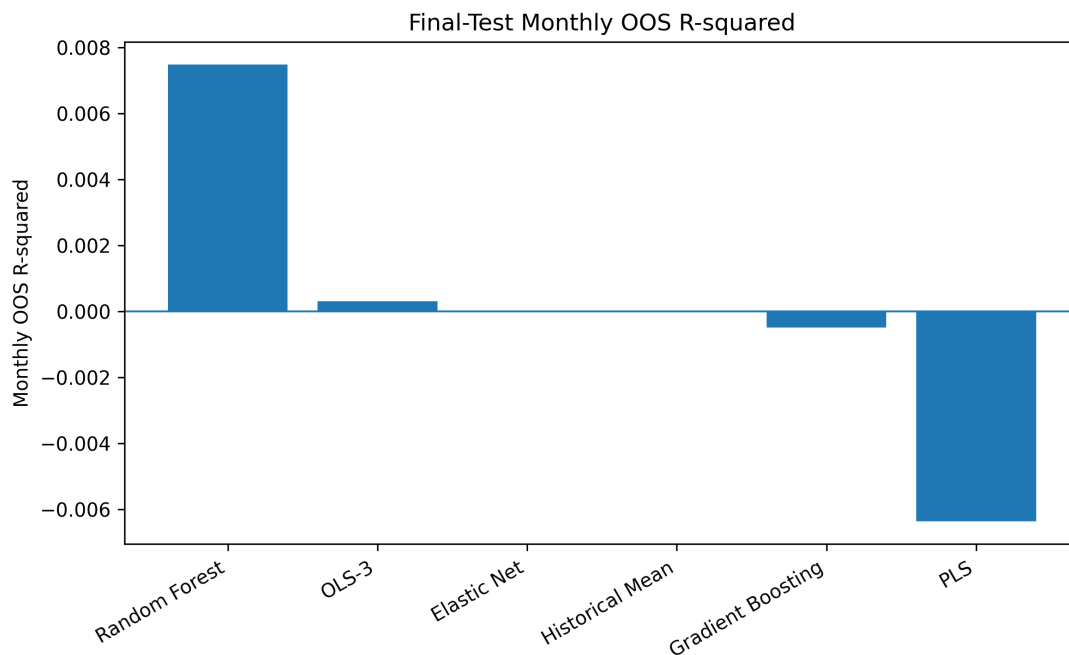


Figure 1: Monthly out-of-sample R^2 relative to the historical-mean benchmark in the 2020–2025 test sample. The graph displays the R^2 values in decimal form, while Table 4 reports them in percent. Random Forest’s value of approximately 0.007482 corresponds to an error reduction of approximately 0.7482%.

Figure 1 makes the benchmark comparison visible. Random Forest has the clearest positive result. OLS-3 is only slightly above zero, Elastic Net overlaps with the historical mean, and PLS and Gradient Boosting are below zero.

The ranking is descriptive. I do not estimate confidence intervals for the differences between models and I do not conduct a multiple-model forecast comparison test. I can therefore say that Random Forest ranks first according to monthly MSE, but I cannot claim that it is statistically significantly better than every alternative.

4.2 Performance across test years

The complete test-period result does not mean that Random Forest performs well in every year. Figure 2 reports the monthly out-of-sample R^2 separately for each of the six test years.

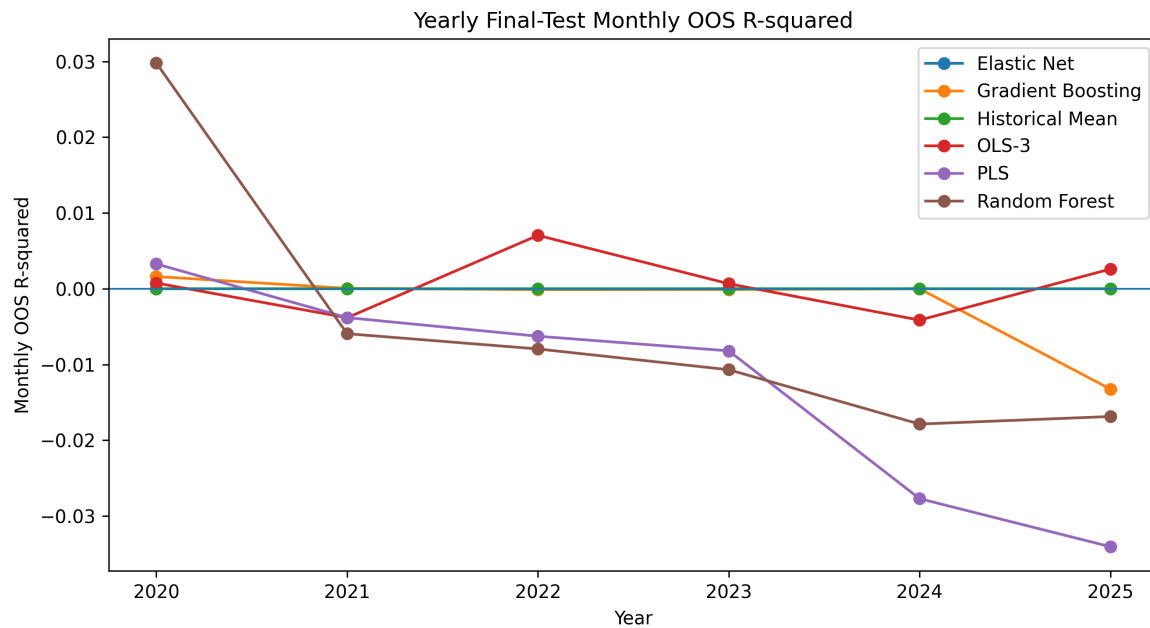


Figure 2: Year-specific monthly out-of-sample R^2 for 2020–2025. Random Forest ranks first over the complete test period but has a positive annual value in only one of the six years. The historical mean and the near-constant Elastic Net predictions overlap close to zero.

Random Forest has a positive annual out-of-sample R^2 only in 2020. Its result in that year is sufficiently strong to offset several negative annual values and produce a positive result over the complete test period.

OLS-3 has a much smaller full-period improvement, but its annual out-of-sample R^2 is positive in four of the six test years. PLS is meaningfully positive in one year and Gradient Boosting in two years when values within 10^{-6} of zero are treated as economically negligible.

This comparison changes how I interpret the overall ranking. Random Forest has the lowest average forecast error over the complete period, but its advantage is concentrated in one year and is not stable across all market environments. OLS-3 produces a much smaller average gain, but its yearly performance is more frequently above the benchmark.

The yearly results therefore support a cautious conclusion. Random Forest performs best according to the main full-period measure, but the evidence does not show that it will dominate the simpler models in every future period.

4.3 What the Random Forest uses

Because Random Forest is the highest-ranked model, I also examine where its fitted prediction variation comes from. Individual predictor names are often technical, especially for the interaction variables, so I first summarize importance by broad predictor group.

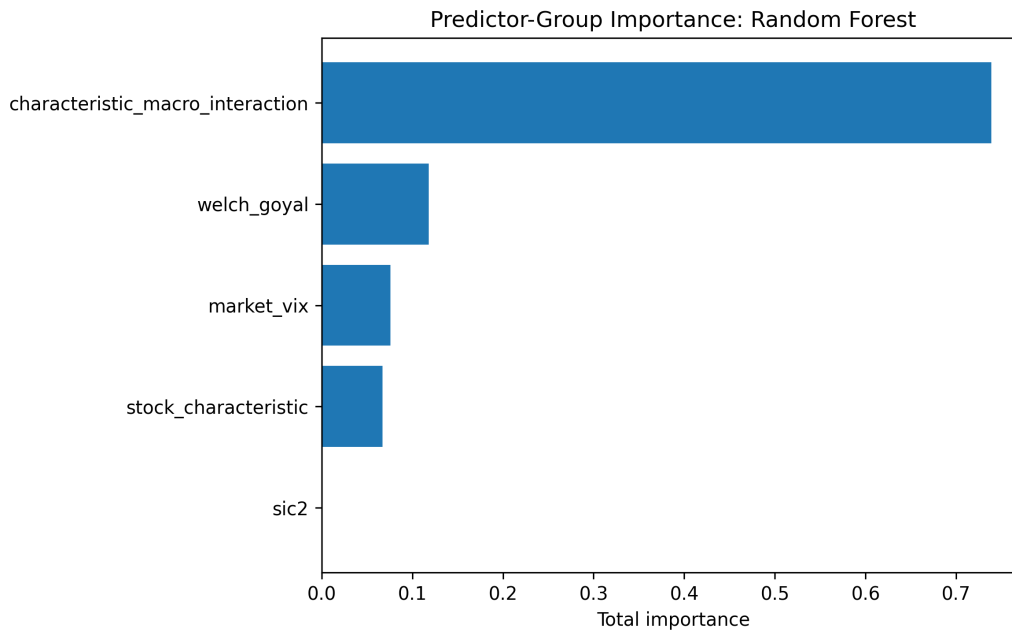


Figure 3: Random Forest impurity importance aggregated by predictor group in the final development-sample fit. The values describe the fitted model and should not be interpreted as causal effects or estimated risk premia.

Characteristic–macro interactions account for approximately 73.9% of the total Random Forest impurity importance. The Welch–Goyal variables account for approximately 11.8%, market and VIX variables for 7.6%, and the remaining stock characteristics for approximately 6.7%. SIC2 industry indicators contribute almost no measured importance.

This pattern suggests that Random Forest obtains much of its fitted prediction variation from relationships that change with aggregate economic conditions. In other words, the model appears to use combinations of firm characteristics and macroeconomic states rather than relying on one single characteristic.

However, this conclusion has to be treated carefully. The interaction group contains 360 predictors, while the other groups contain far fewer variables. Its high total importance therefore does not mean that every interaction is individually important.

Impurity importance also has general limitations. Correlated variables can divide importance among themselves, and the values can depend on the number of possible split points. The figure describes how the fitted Random Forest makes predictions; it does not identify causal effects.

4.4 Comparison with the machine-learning asset-pricing literature

The general motivation of this project follows the machine-learning asset-pricing literature. Flexible models may be useful because they can handle many predictors and allow nonlinear relationships and interactions (Gu et al. 2020).

The results of this project are mixed. Random Forest improves on the historical-mean benchmark, while Gradient Boosting and PLS do not. OLS-3 also remains competitive despite using only three predictors. The results therefore support the idea that flexible models can help, but they also show that greater complexity is not always rewarded outside the estimation sample.

The numerical results should not be directly compared with percentages reported in another study. The stock universe, sample period, predictors, preprocessing, and benchmark can differ. In particular, the out-of-sample R^2 in this project uses the estimated historical mean as its benchmark, rather than a zero-return benchmark.

4.5 Financial interpretation

The project compares statistical forecast accuracy. It does not evaluate a trading strategy.

A higher out-of-sample R^2 does not automatically imply a higher Sharpe ratio or a profitable long-short portfolio. A model may reduce average squared prediction error without producing a useful ranking after trading costs and portfolio constraints are considered.

The current analysis does not include:

- portfolio formation;
- transaction costs;
- portfolio turnover;
- liquidity or market-impact costs;
- short-selling restrictions;
- risk-adjusted portfolio returns;
- Sharpe ratios or cumulative trading returns.

The financial conclusion is therefore limited. Random Forest has the lowest forecast error in the 2020–2025 test period, but the project does not show that trading on its predictions would be profitable.

A portfolio analysis would be a separate extension. It would require a predefined sorting or weighting rule and an evaluation of both gross and net returns. It should not be designed after examining the same test results, because that would use the test sample for a second model-selection exercise.

5 Critical Reflection

This project involved more than estimating and comparing six models. A large part of the work was identifying problems in the data pipeline, checking whether the timing rules were realistic, and deciding which results could be treated as final. In this section, I discuss the main problems I encountered, how I dealt with them, and which limitations remain despite the corrections.

5.1 Problems encountered and how I addressed them

Table 5: Main project problems, solutions, and remaining limitations

Problem	Action taken	How far the solution goes
Nonconsecutive return targets	An earlier target construction used the next available observation for each ticker. This could incorrectly treat a return across a gap of several months as a one-month return. I added a condition requiring the next observation to be the exact following calendar month and rebuilt the final panel.	This fully solves the calendar-gap problem inside the available monthly histories. It does not recover returns for months that are completely missing or delisting returns that are unavailable in Yahoo Finance.
Timing of Welch–Goyal variables	An earlier version merged Welch–Goyal variables in the same month as the stock predictors. I shifted these variables by one month so that stock month t uses the macroeconomic state from month $t - 1$.	This removes the identified same-month timing problem and makes the rule more conservative. It does not reproduce the exact historical publication date of every macroeconomic series.
Timing of accounting information	Compustat <code>datadate</code> is the fiscal-year-end date, not the publication date. I treated each report as available six months after fiscal-year end, merged it backward to the most recent available report, and prevented very old reports from being carried forward indefinitely.	The procedure substantially reduces look-ahead bias. However, the six-month delay is still an approximation because exact filing dates are not used.
Missing and extreme predictor values	I replaced missing continuous characteristics with the cross-sectional median of the same month and winsorized these characteristics at the monthly 1st and 99th percentiles. I also applied transparent daily and monthly quality filters to remove unusable price histories.	The final panel contains no missing predictors, infinite values, or duplicate ticker-month observations. Winsorization limits extreme predictor values, but the return target remains raw and still contains very large observations.
Inconsistent documentation	Earlier documentation described rank normalization, 499 predictors, and an active Decision Tree. These statements no longer matched the final code. I used the final repository audit and the authoritative output files to reconcile the paper with the implemented pipeline.	This solves the inconsistency once the paper, README, and Data Construction Guide are all updated. It does not change the numerical results, but it is important for making the project understandable and reproducible.
Computational cost of tree tuning	Gradient Boosting tuning was very slow because the model fits trees sequentially across 15 annual folds. I saved every completed fold and candidate, verified the 450 fold results, and promoted the completed tuning outputs into the official five-step workflow without repeating the expensive fits.	The official code can reproduce the full tuning in the future, and provenance metadata records the migration. Per-fold runtime and warning diagnostics were not recorded by the completed temporary run.

The first two issues were particularly important because they could create look-ahead bias or an incorrect target. After correcting them, I rebuilt the final data and repeated the later modeling stages. The final panel therefore uses only consecutive one-month targets and one-month-lagged Welch–Goyal states.

The accounting solution is less complete. The six-month lag is a practical and conservative rule, but it does not provide the same precision as exact filing dates. I therefore treat this as a limitation that has been reduced rather than fully removed.

5.2 What worked well

The strongest methodological decision was the chronological sample design. I used expanding training windows instead of random cross-validation and required every candidate to predict 15 future calendar years. Including the original fixed specifications directly in the grids made the rule simple and auditable: the exact minimum average monthly MSE determines the selected configuration.

The revision also clarified an important limitation. Although the selection code never loads test observations, the 2020–2025 outcomes had already been observed when the earlier implementation was

reviewed. I therefore report the current analysis as a corrected revised result rather than describing the test period as newly opened or untouched during the revision process.

The use of a historical-mean benchmark also worked well. It made the interpretation direct: a model only provides value under the chosen loss function if it reduces forecast error relative to a simple constant prediction.

The project also produced useful negative results. Elastic Net selected no variables, PLS underperformed the benchmark, and Gradient Boosting remained slightly below it. These findings are informative because they show that large predictor sets, regularization, dimension reduction, and nonlinear models do not automatically generate better forecasts.

5.3 What the model failures taught me

Elastic Net shrinks all 484 coefficients to zero. At first, this result looked like a possible implementation error. After checking the fitted coefficients, predictions, tuning results, and benchmark, I concluded that it is a valid model outcome. Under the selected penalty, the model preferred a constant forecast rather than unstable linear effects.

This result does not mean that all stock characteristics are always irrelevant. It means that the selected Elastic Net specification did not find sufficiently stable linear information to improve on the historical mean under this sample and chronology.

PLS presents a different failure. Reducing the 484 predictors to two supervised components improved expanding-window performance relative to the fixed 20-component version. Nevertheless, it underperformed the benchmark in the final test. This shows that dimension reduction can control complexity without guaranteeing that the estimated components remain useful in a later market period.

Gradient Boosting also illustrates the danger of model flexibility. The model sequentially corrects earlier errors, but these corrections may fit unstable patterns. Its test out-of-sample R^2 is slightly negative and its prediction-target correlation is close to zero, suggesting little stable incremental information in the test period.

Random Forest is the only complex model with a clear positive full-period result. Averaging many randomized trees appears to be more effective than sequential boosting in this sample. However, its success is not stable across years, so I do not interpret it as a universal relationship.

5.4 Limits of the Random Forest result

Random Forest reduces average monthly squared forecast error by approximately 0.75%, but it has a positive annual out-of-sample R^2 in only one of the six test years. Its strong performance in 2020 has a large effect on the full-period ranking.

OLS-3 produces a much smaller total improvement, but it is positive in four of the six years. This comparison shows that there are at least two ways to assess a model:

- its average performance over the full test period;
- the stability of its performance across shorter periods.

Random Forest performs better according to the first measure, while OLS-3 is more frequently above the benchmark. Therefore, the final ranking should not be interpreted as evidence that Random Forest will always be the best choice.

The target distribution also makes the ranking sensitive to extreme errors. The raw next-month return contains a small number of very large observations, and squared loss gives these observations high weight. Reporting MAE provides a useful comparison, but the primary ranking still depends on MSE.

5.5 Data limitations that remain

The stock universe is based on a locked S&P 500-style ticker list rather than a fully point-in-time index-constituent database. The locked list ensures that the same universe is used throughout the pipeline, but it does not fully solve survivorship and selection bias.

Ticker-based matching can also be affected by:

- ticker changes;
- mergers and acquisitions;
- delistings;
- incomplete historical membership information;
- differences between Yahoo and Compustat identifiers.

Yahoo Finance is convenient and provides a long price history, but it does not include a permanent security identifier, complete delisting returns, or all the market microstructure information available from CRSP. Adjusted prices may also be revised.

The accounting variables are annual, while market variables are monthly or constructed from daily data. Accounting information therefore changes much less frequently. Some constructed characteristics are also practical proxies rather than exact replications of every definition used in the literature.

The 360 characteristic-macro interactions provide flexibility, but they also create many correlated predictors. Their high Random Forest group importance partly reflects that this group contains far more variables than the other groups. It should therefore not be interpreted as proof that every interaction is economically important.

5.6 Evaluation limitations

The test period contains only six years. Although it includes several different market conditions, it is still a relatively short period for evaluating the stability of stock-return prediction.

I also do not conduct a formal statistical comparison of forecast losses. The ranking shows that Random Forest has the lowest monthly MSE, but it does not show that the difference is statistically significant after accounting for time dependence and the comparison of several models.

The current out-of-sample R^2 evaluates forecast levels relative to the historical mean. An additional cross-sectional measure could remove each month's average return and average prediction. This would test more directly whether a model can identify which stocks perform relatively better or worse within each month.

Finally, the project evaluates prediction error and not an investment strategy. It does not report:

- long-short portfolio returns;
- Sharpe ratios;
- transaction costs;
- turnover;
- market impact;
- short-selling constraints;
- factor-adjusted performance.

A lower MSE therefore cannot automatically be interpreted as a profitable trading opportunity.

5.7 What I would do differently

With more time and better data access, I would improve the project in the following order.

I would use a point-in-time CRSP/Compustat database with permanent identifiers, complete delisting returns, and historical constituent information. I would also use exact accounting filing dates instead of a fixed six-month lag.

6 Conclusion

This project examined whether machine-learning methods can improve the prediction of next-month stock excess returns relative to a historical-mean benchmark. I compared six approaches that represent different levels of model complexity: the Historical Mean, OLS-3, Partial Least Squares, Elastic Net, Random Forest, and Gradient Boosting.

The results provide a cautious answer to the research question. Random Forest ranks first according to the main monthly MSE criterion. Its monthly out-of-sample R^2 is 0.7482%, meaning that it reduces average monthly squared prediction error by approximately 0.75% relative to the historical mean. This is a positive result, but it is still a modest improvement and should not be interpreted as an investment return.

OLS-3 ranks second and produces only a very small improvement over the benchmark. Nevertheless, its result is important because it uses only size, book-to-market, and 12-month momentum. The fact that such a simple model remains competitive shows that increasing the number of predictors and the complexity of the model does not automatically lead to better forecasts.

The other models provide useful negative results. Elastic Net shrinks all 484 coefficients to zero and therefore produces almost the same forecast as the historical mean. PLS performs worse than the benchmark even after selecting two components. Gradient Boosting remains slightly below the benchmark, although its depth-1 configuration improves substantially on the earlier depth-2 result. These findings show that regularization, dimension reduction, and nonlinear modeling are not sufficient on their own. Their success depends on whether the relationships estimated in the training period remain stable in later periods.

The yearly analysis also changes the interpretation of the overall ranking. Random Forest has a positive annual out-of-sample R^2 in only one of the six test years, while OLS-3 is positive in four years. The Random Forest result is therefore driven strongly by its performance in 2020. I conclude that Random Forest has the best average test-period performance, but not that it is the most reliable model in every market environment.

An important lesson from the project is that the research design matters as much as the model itself. Fifteen expanding validation years, fold-local preprocessing, an exact selection rule, and a simple benchmark reduce the scope for selecting a model only because it fits one period well. The revised design selects RF-300 and the shallow GB-100 configuration directly from the common candidate grids. Because the test outcomes had already been observed when the revision was undertaken, genuinely new extensions require a new future holdout.

The project evaluates statistical forecast accuracy rather than portfolio performance. The results do not show whether the predictions can generate profitable long-short portfolios after turnover, transaction costs, liquidity constraints, and short-selling restrictions. Therefore, the financial conclusion is limited to forecast performance: Random Forest reduces prediction error modestly, but the project does not establish a profitable trading strategy.

A natural extension would be to add neural networks. Neural networks could capture more complex nonlinear relationships among stock characteristics, market variables, and macroeconomic states. However, they would also introduce more hyperparameters and a greater risk of overfitting. I would therefore begin with a relatively shallow network and use strong regularization, early stopping, and the same expanding-window validation design used in the current project.

Because the data contain 484 predictors, I would also examine a PCA–neural-network approach. Principal Component Analysis could first reduce the predictor set to a smaller number of components, and the neural network could then model nonlinear relationships among those components. This may reduce computational cost, noise, and multicollinearity. At the same time, PCA is unsupervised and

may discard variables that explain little predictor variance but still contain useful return information. I would therefore compare three alternatives:

- a neural network using the complete predictor set;
- a neural network using PCA components;
- a neural network using supervised components or selected predictor groups.

The comparison would have to use the same chronological samples and a genuinely new test period. I would not select the neural-network architecture based on the 2020–2025 results that have already been observed.

Further extensions would include cross-sectional out-of-sample R^2 , formal tests of differences in prediction loss, and a pre-specified portfolio evaluation with transaction costs. Better data would also improve the study, especially a point-in-time CRSP/Compustat universe with permanent identifiers, delisting returns, and exact accounting filing dates.

Overall, the project does not show that machine learning solves the stock-return prediction problem. It shows that one ensemble method can produce a modest improvement, while several other sophisticated methods fail to outperform a simple benchmark. The main conclusion is therefore that model complexity can be useful, but only when it is combined with careful timing, chronological validation, realistic benchmarks, and cautious interpretation.

References

- Breiman, L. (2001). “Random Forests”. In: *Machine Learning* 45.1, pp. 5–32.
- Fama, E. F. and K. R. French (1992). “The Cross-Section of Expected Stock Returns”. In: *The Journal of Finance* 47.2, pp. 427–465.
- Gu, S., B. Kelly, and D. Xiu (2020). “Empirical Asset Pricing via Machine Learning”. In: *The Review of Financial Studies* 33.5, pp. 2223–2273.
- Jegadeesh, N. and S. Titman (1993). “Returns to Buying Winners and Selling Losers: Implications for Stock Market Efficiency”. In: *The Journal of Finance* 48.1, pp. 65–91.
- Tibshirani, R. (1996). “Regression Shrinkage and Selection via the Lasso”. In: *Journal of the Royal Statistical Society: Series B* 58.1, pp. 267–288.
- Welch, I. and A. Goyal (2008). “A Comprehensive Look at the Empirical Performance of Equity Premium Prediction”. In: *The Review of Financial Studies* 21.4, pp. 1455–1508.
- Wold, S. et al. (1984). “The Collinearity Problem in Linear Regression: The Partial Least Squares Approach to Generalized Inverses”. In: *SIAM Journal on Scientific and Statistical Computing* 5.3, pp. 735–743.
- Zou, H. and T. Hastie (2005). “Regularization and Variable Selection via the Elastic Net”. In: *Journal of the Royal Statistical Society: Series B* 67.2, pp. 301–320.

A Code and Reproducibility Overview

The complete implementation is provided in the accompanying repository. The active model pipeline contains five stages:

1. fixed baseline estimation;
2. 15-fold expanding-window tuning;
3. exact-minimum configuration selection;
4. development-sample refitting and test evaluation;
5. interpretation tables, workbook, and figures.

The authoritative execution order from the repository root is:

```
python3 src/models/step_01_fixed/01_fixed_linear_models.py
python3 src/models/step_01_fixed/02_fixed_tree_models.py
python3 src/models/step_01_fixed/03_compare_fixed_models.py
python3 src/models/step_02_tuning/01_tune_linear_models.py
python3 src/models/step_02_tuning/02_tune_tree_models.py
python3 src/models/step_02_tuning/03_compare_tuning_results.py
python3 src/models/step_03_selection/01_select_configurations.py
python3 src/models/step_04_test/01_final_test_evaluation.py
python3 src/models/step_05_interpretation/run_all.py
```

For the present migration, the expensive Step 02 fits were not repeated. The completed verified fold and candidate results were validated and promoted into the official Step 02 output schema; execution then began at Step 03. Migration provenance is stored in `output/models/tuning/tuning_migration_provenance.json`. The official Step 02 source remains the procedure for a complete future reproduction.